PROYECTO 2.º PARCIAL

Perceptrón Monocapa (SLP)
Compuerta lógica AND

Fase 1: procedimiento matemático
Fase 2: aplicación web interactiva con Python y Streamlit

Integrantes: _____

Asignatura: Inteligencia Artificial / Ciencia de Datos

Fecha: _____

1. Objetivo

Desarrollar una aplicación web que simule y explique el aprendizaje de un perceptrón monocapa para resolver la compuerta AND. Se muestran los cálculos muestra por muestra, el historial por épocas, inferencias y exportación a PDF.

2. Parámetros y datos de entrenamiento

Valores iniciales: w1=0.1, w2=0.3, b=0.2, tasa de aprendizaje η=0.1. Máximo: cuatro épocas.

x1	x2	Objetivo
0	0	0
0	1	0
1	0	0
1	1	1

3. Regla matemática

Suma ponderada: $z=(x1\cdot w1)+(x2\cdot w2)+b$.

Escalón: $y=1$ si $z\geq 0$; $y=0$ si $z<0$.

Error: $e=t-y$.

Actualización: $w1'=w1+\eta\cdot e\cdot x1$; $w2'=w2+\eta\cdot e\cdot x2$; $b'=b+\eta\cdot e$.

El bias se actualiza como si existiera una entrada constante igual a 1. Las muestras se procesan en el orden de la tabla y los parámetros se actualizan después de cada muestra.

4. Fase 1 — Cálculos completos

Cada fila muestra la suma ponderada calculada con los parámetros vigentes, la salida, el error y los parámetros nuevos.

Época 1

Entrada	t	z	y	e	w1 nuevo	w2 nuevo	b nuevo
(0,0)	0	0.2000	1	-1	0.1000	0.3000	0.1000
(0,1)	0	0.4000	1	-1	0.1000	0.2000	0.0000
(1,0)	0	0.1000	1	-1	0.0000	0.2000	-0.1000
(1,1)	1	0.1000	1	0	0.0000	0.2000	-0.1000

Fin de época: w1=0.0000, w2=0.2000, b=-0.1000; errores=3.

Época 2

Entrada	t	z	y	e	w1 nuevo	w2 nuevo	b nuevo
(0,0)	0	-0.1000	0	0	0.0000	0.2000	-0.1000
(0,1)	0	0.1000	1	-1	0.0000	0.1000	-0.2000
(1,0)	0	-0.2000	0	0	0.0000	0.1000	-0.2000
(1,1)	1	-0.1000	0	1	0.1000	0.2000	-0.1000

Fin de época: w1=0.1000, w2=0.2000, b=-0.1000; errores=2.

Época 3

Entrada	t	z	y	e	w1 nuevo	w2 nuevo	b nuevo
(0,0)	0	-0.1000	0	0	0.1000	0.2000	-0.1000

(0,1)	0	0.1000	1	-1	0.1000	0.1000	-0.2000
(1,0)	0	-0.1000	0	0	0.1000	0.1000	-0.2000
(1,1)	1	-0.0000	0	1	0.2000	0.2000	-0.1000

Fin de época: w1=0.2000, w2=0.2000, b=-0.1000; errores=2.

Época 4

Entrada	t	z	y	e	w1 nuevo	w2 nuevo	b nuevo
(0,0)	0	-0.1000	0	0	0.2000	0.2000	-0.1000
(0,1)	0	0.1000	1	-1	0.2000	0.1000	-0.2000
(1,0)	0	0.0000	1	-1	0.1000	0.1000	-0.3000
(1,1)	1	-0.1000	0	1	0.2000	0.2000	-0.2000

Fin de época: w1=0.2000, w2=0.2000, b=-0.2000; errores=3.

5. Resumen por épocas

Época	w1 final	w2 final	Bias final	Errores
1	0.0000	0.2000	-0.1000	3
2	0.1000	0.2000	-0.1000	2
3	0.2000	0.2000	-0.1000	2
4	0.2000	0.2000	-0.2000	3

Interpretación

Con la función escalón definida como y=1 cuando z≥0, el entrenamiento no converge dentro de las cuatro épocas permitidas. La aplicación conserva y muestra los errores y la evolución de los parámetros hasta el límite solicitado; por ello, no debe afirmarse que el modelo ya aprendió correctamente AND después de cuatro épocas.

6. Explicación de los módulos y funciones

perceptron.py: AND_DATA guarda las cuatro muestras. funcion_escalon(z) calcula la salida binaria. entrenar(...) recorre épocas y muestras, calcula z, y, error, actualiza pesos y bias y almacena el historial. predecir(...) evalúa una combinación con los parámetros finales.

pdf_generator.py: generar_pdf(resultado, parametros) crea un PDF con parámetros, resumen por épocas y operaciones detalladas; devuelve el archivo en memoria para que Streamlit lo descargue.

app.py: configura la página, dibuja el esquema de red, captura hiperparámetros, inicia entrenamiento, presenta tablas y cálculos LaTeX, permite inferencia y ofrece el botón de descarga.

requirements.txt: contiene las bibliotecas necesarias para instalar el proyecto.

7. Pantallas de la aplicación

Estas son maquetas de referencia de las vistas previstas en la aplicación; las capturas reales se pueden tomar al ejecutar Streamlit en el navegador.

Perceptrón AND

Pantalla 1 — esquema e hiperparámetros

Esquema: x1 ■■ w1 ■■■ x2 ■■ w2 ■■■■■ Σ + b ■■■ Escalón ■■■ y

Parámetros: w1 [0.100] w2 [0.300] Bias [0.200] η [0.100] Épocas [4]

Datos AND: (0,0)→0 (0,1)→0 (1,0)→0 (1,1)→1

Botón: [Entrenar perceptrón]

Entrenamiento completado

Pantalla 2 — resumen y procedimiento

Resumen por épocas: tabla con época, w1 final, w2 final, bias final y errores.

Procedimiento: paneles desplegables por muestra con z, salida, error y actualización de w1, w2 y bias.

Exportación: [Descargar informe PDF del entrenamiento]

Módulo de inferencia / prueba

Pantalla 3 — evaluación de entradas

Entrada x1: [0 ▼] **Entrada x2:** [1 ▼]

Salida predicha: 0 **Suma ponderada:** z = ...

Al cambiar las entradas se actualiza la predicción con los parámetros finales.

8. Instalación y ejecución

1. Instalar Python 3.9 o superior.
2. Abrir una terminal dentro de la carpeta del proyecto.
3. (Opcional) Crear y activar un entorno virtual.
4. Ejecutar **pip install -r requirements.txt**.
5. Ejecutar **streamlit run app.py**.
6. Abrir la dirección local que aparece en la terminal, normalmente http://localhost:8501.

9. Evidencia del código fuente

A continuación se incluyen capturas tipográficas del código principal. Los archivos completos se encuentran en el ZIP adjunto.

Código fuente: perceptron.py

```
"""Lógica de entrenamiento e inferencia del perceptrón AND."""

AND_DATA = [
    (0, 0, 0),
    (0, 1, 0),
    (1, 0, 0),
    (1, 1, 1),
]

def funcion_escalon(z):
    """Devuelve 1 si z >= 0; de lo contrario devuelve 0."""
    return 1 if z >= 0 else 0

def entrenar(w1, w2, b, eta, max_epocas, datos=None):
    """
    Entrena muestra por muestra.
    Actualiza w1, w2 y b con:
      w1 = w1 + eta * error * x1
      w2 = w2 + eta * error * x2
      b  = b  + eta * error
    """
    if datos is None:
        datos = AND_DATA

    historial_pasos = []
    resumen_epocas = []

    for epoca in range(1, max_epocas + 1):
        errores_epoca = 0

        for x1, x2, objetivo in datos:
            z = x1 * w1 + x2 * w2 + b
            salida = funcion_escalon(z)
            error = objetivo - salida

            w1_anterior, w2_anterior, b_anterior = w1, w2, b

            w1 = w1 + eta * error * x1
            w2 = w2 + eta * error * x2
            b = b + eta * error

            if error != 0:
                errores_epoca += 1
```

Código fuente: perceptron.py (continuación)

```
            historial_pasos.append({
                "epoca": epoca, "x1": x1, "x2": x2,
                "objetivo": objetivo, "z": z, "salida": salida,
                "error": error, "w1_antes": w1_anterior,
                "w2_antes": w2_anterior, "b_antes": b_anterior,
                "w1_despues": w1, "w2_despues": w2, "b_despues": b
            })

        resumen_epocas.append({
            "epoca": epoca, "w1": w1, "w2": w2,
            "b": b, "errores": errores_epoca
        })

        # Se puede detener antes si ya clasificó correctamente las 4 muestras.
        if errores_epoca == 0:
            break

    return {
        "w1": w1, "w2": w2, "b": b,
        "pasos": historial_pasos,
        "epocas": resumen_epocas
    }

def predecir(x1, x2, w1, w2, b):
    """Evalúa una combinación de entradas con los parámetros aprendidos."""
    z = x1 * w1 + x2 * w2 + b
    return {"x1": x1, "x2": x2, "z": z, "salida": funcion_escalon(z)}
```

Código fuente: pdf_generator.py

```
"""Generación del informe PDF del entrenamiento."""
from io import BytesIO
from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.units import mm
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle

def generar_pdf(resultado, parametros):
    buffer = BytesIO()
    doc = SimpleDocTemplate(
        buffer, pagesize=A4, rightMargin=16*mm, leftMargin=16*mm,
        topMargin=16*mm, bottomMargin=16*mm
    )
    estilos = getSampleStyleSheet()
    elementos = [
        Paragraph("Informe de entrenamiento: Perceptrón AND", estilos["Title"]),
        Paragraph("Perceptrón monocapa (SLP) — función de activación escalón", estilos["Heading2"]),
        Spacer(1, 8),
        Paragraph(
            f"Parámetros iniciales: w1={parametros['w1']}, "
            f"w2={parametros['w2']}, b={parametros['b']}, "
            f"η={parametros['eta']}. Máximo de épocas: {parametros['max_epocas']}.",
            estilos["BodyText"]
        ),
        Spacer(1, 8),
        Paragraph("Regla matemática", estilos["Heading2"]),
        Paragraph("z = (x1 × w1) + (x2 × w2) + b;  y = 1 si z ≥ 0, en otro caso y = 0.", estilos["BodyText"]),
        Paragraph("e = objetivo − y;  w1' = w1 + η·e·x1;  w2' = w2 + η·e·x2;  b' = b + η·e.", estilos["BodyText"]),
        Spacer(1, 10),
        Paragraph("Resumen por épocas", estilos["Heading2"])
    ]
    tabla = [["Época", "w1 final", "w2 final", "Bias final", "Errores"]]
    for r in resultado["epocas"]:
        tabla.append([str(r["epoca"]), f'{r["w1"]:.4f}', f'{r["w2"]:.4f}', f'{r["b"]:.4f}', str(r["errores"])])
    t = Table(tabla, repeatRows=1, hAlign="LEFT")
    t.setStyle(TableStyle([
        ("BACKGROUND", (0,0), (-1,0), colors.HexColor("#173B57")),
        ("TEXTCOLOR", (0,0), (-1,0), colors.white),
        ("GRID", (0,0), (-1,-1), .5, colors.HexColor("#B8C7D1")),
        ("ROWBACKGROUNDS", (0,1), (-1,-1), [colors.white, colors.HexColor("#EEF4F7")]),
        ("PADDING", (0,0), (-1,-1), 7),
    ]))
```

Código fuente: pdf_generator.py (continuación)

```
    elementos += [t, Spacer(1, 10), Paragraph("Procedimiento muestra por muestra", estilos["Heading2"])]
    pasos = [["Época", "Entrada", "t", "z", "y", "e", "w1 nuevo", "w2 nuevo", "b nuevo"]]
    for p in resultado["pasos"]:
        pasos.append([
            str(p["epoca"]), f'({p["x1"]},{p["x2"]})', str(p["objetivo"]),
            f'{p["z"]:.4f}', str(p["salida"]), str(p["error"]),
            f'{p["w1_despues"]:.4f}', f'{p["w2_despues"]:.4f}', f'{p["b_despues"]:.4f}'
        ])
    t2 = Table(pasos, repeatRows=1, colWidths=[13*mm, 17*mm, 9*mm, 14*mm, 9*mm, 9*mm, 19*mm, 19*mm, 19*mm])
    t2.setStyle(TableStyle([
        ("BACKGROUND", (0,0), (-1,0), colors.HexColor("#173B57")),
        ("TEXTCOLOR", (0,0), (-1,0), colors.white),
        ("GRID", (0,0), (-1,-1), .35, colors.HexColor("#AAB8C2")),
        ("FONTSIZE", (0,0), (-1,-1), 7),
        ("PADDING", (0,0), (-1,-1), 4),
        ("ROWBACKGROUNDS", (0,1), (-1,-1), [colors.white, colors.HexColor("#F2F6F8")]),
    ]))
    elementos.append(t2)
    doc.build(elementos)
    buffer.seek(0)
    return buffer.getvalue()
```

Código fuente: app.py

```
import streamlit as st
import pandas as pd
from perceptron import entrenar, predecir, AND_DATA
from pdf_generator import generar_pdf

st.set_page_config(page_title="Perceptrón AND", page_icon="■", layout="wide")
st.title("■ Simulador del Perceptrón Monocapa")
st.caption("Entrenamiento e inferencia de la compuerta lógica AND")

st.subheader("1. Esquema de la red neuronal")
st.markdown("""
<div style="background:#eef4f8;padding:16px;border-radius:12px;text-align:center;color:#173b57">
<b>Entradas</b>   x■ ■■(w■)■■■<br>
<b></b>   x■ ■■(w■)■■■■■■ <b>Σ + b</b> ■■■ <b>Función escalón</b> ■■■ <b>Salida y</b><br>
<b></b>   Bias b ■■■■■■■■■■
</div>
""", unsafe_allow_html=True)

st.subheader("2. Hiperparámetros iniciales")
c1, c2, c3, c4, c5 = st.columns(5)
with c1:
    w1 = st.number_input("Peso w1", value=0.1, step=0.1, format="%.3f")
with c2:
    w2 = st.number_input("Peso w2", value=0.3, step=0.1, format="%.3f")
with c3:
    bias = st.number_input("Bias b", value=0.2, step=0.1, format="%.3f")
with c4:
    eta = st.number_input("Aprendizaje η", min_value=0.001, value=0.1, step=0.01, format="%.3f")
with c5:
    max_epocas = st.number_input("Máximo de épocas", min_value=1, max_value=100, value=4, step=1)

st.markdown("**Conjunto de entrenamiento AND**")
st.dataframe(pd.DataFrame(AND_DATA, columns=["x1", "x2", "Objetivo"]), hide_index=True, use_container_width=True)

if st.button("■ Entrenar perceptrón", type="primary"):
    st.session_state["resultado"] = entrenar(w1, w2, bias, eta, int(max_epocas))
    st.session_state["parametros"] = {
        "w1": w1, "w2": w2, "b": bias, "eta": eta,
        "max_epocas": int(max_epocas)
    }

if "resultado" in st.session_state:
    resultado = st.session_state["resultado"]
```

Código fuente: app.py (continuación)

```
    parametros = st.session_state["parametros"]
    st.success(f"Entrenamiento terminado: {len(resultado['epocas'])} época(s) ejecutada(s).")
    st.subheader("3. Resumen por épocas")
    df_epocas = pd.DataFrame(resultado["epocas"]).rename(columns={
        "epoca": "Época", "w1": "w1 final", "w2": "w2 final",
        "b": "Bias final", "errores": "Errores"
    })
    st.dataframe(df_epocas, hide_index=True, use_container_width=True)

    st.subheader("4. Procedimiento matemático paso a paso")
    for p in resultado["pasos"]:
        with st.expander(f"Época {p['epoca']} — entrada ({p['x1']}, {p['x2']})"):
            st.latex(
                rf"z = ({p['x1']} \cdot {p['w1_antes']:.4f}) + "
                rf"({p['x2']} \cdot {p['w2_antes']:.4f}) + {p['b_antes']:.4f}"
                rf" = {p['z']:.4f}"
            )
            st.latex(rf"y=f(z)={p['salida']},\quad e=t-y={p['objetivo']}-{p['salida']}={p['error']}")
            st.latex(
                rf"w_1'={p['w1_antes']:.4f}+{eta}\cdot({p['error']})\cdot{p['x1']}={p['w1_despues']:.4f}"
            )
            st.latex(
                rf"w_2'={p['w2_antes']:.4f}+{eta}\cdot({p['error']})\cdot{p['x2']}={p['w2_despues']:.4f}"
            )
            st.latex(
                rf"b'={p['b_antes']:.4f}+{eta}\cdot({p['error']})={p['b_despues']:.4f}"
            )

    st.subheader("5. Módulo de inferencia / prueba")
    st.write("Selecciona una combinación de entradas para obtener la predicción con los parámetros finales.")
    a, bcol = st.columns(2)
    with a:
        x1_test = st.selectbox("Entrada x1", [0, 1], key="x1_test")
    with bcol:
        x2_test = st.selectbox("Entrada x2", [0, 1], key="x2_test")
    pred = predecir(x1_test, x2_test, resultado["w1"], resultado["w2"], resultado["b"])
    st.metric("Salida predicha", pred["salida"])
    st.write(f"Suma ponderada: z = {pred['z']:.4f}")
    st.caption("La salida se actualiza al cambiar cualquiera de las entradas.")

    pdf_bytes = generar_pdf(resultado, parametros)
    st.download_button(
        "■ Descargar informe PDF del entrenamiento",
```

Código fuente: app.py (continuación)

```
        data=pdf_bytes,
        file_name="informe_perceptron_and.pdf",
        mime="application/pdf"
    )
else:
    st.info("Configura los parámetros y presiona «Entrenar perceptrón» para ver los cálculos, resultados e inferencia.")
```

10. Conclusión

El proyecto integra el procedimiento matemático del perceptrón AND con una aplicación web modular. La interfaz permite observar las operaciones, estudiar la evolución de los parámetros, probar entradas y exportar un informe. Para una entrega con evidencia definitiva, se recomienda ejecutar la aplicación y añadir capturas reales del navegador en sustitución o complemento de las maquetas de referencia.